Python Indexing, Slicing, nditer, and Copies Quiz

Part 1: Indexing & Slicing (Core Python & NumPy)

1. What happens when you attempt to access `my_string[len(my_string)]` in Python?

A) It returns the last character of the string.

B) It returns an empty string.

C) It raises an `IndexError`

. D) It returns `None`.

2. Given the list `arr = [10, 20, 30, 40, 50]`, what is the output of `arr[-4]`?

A) 10

B) 20

C) 40

D) 50

3. In NumPy, given `matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])`, what does `matrix[1, 2]` return?

A) 2

B) 6

C) 8

D) `[4, 5, 6]`

4. Given the list `nums = [0, 1, 2, 3, 4, 5, 6]`, what is the result of `nums[1:6:2]`?

A) `[1, 2, 3, 4, 5, 6]`

B) `[1, 3, 5]`

C) `[2, 4, 6]`

D) `[0, 2, 4]`

5. What is the output of `word = "Python"; print(word[::-1])`?

A) Python

B) nohtyP

C) P

D) n

6. Given the list `data = [5, 10, 15, 20]`, what happens if you execute `data[1:100]`?

A) It raises an `IndexError`.

B) It returns `[10, 15, 20]`.

C) It pads the remaining slots with `None`.

D) It returns a `MemoryError`.

7. Which of the following statements about string indexing in Python is true?

A) You can change a specific character by assigning it a new value, like `s[0] = 'A'`.

B) String indexing is 1-based.

C) Strings do not support negative indexing.

D) Strings are immutable, so item assignment like `s[0] = 'A'` raises a `TypeError`.

8. Given `arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])`, what does `arr[:2, 1:]` return?

A) `[[2, 3], [5, 6]]`

B) `[[4, 5], [7, 8]]`

C) `[[1, 2], [4, 5]]`

D) `[[5, 6], [8, 9]]`

9. Given `x = [1, 2, 3]`, which of the following operations creates a shallow copy of the entire list using slicing?

A) `y = x[0:]`

B) `y = x[:]`

C) `y = x`

D) Both A and B are correct.

10. Given `text = "Programming"`, what is the output of `text[-4:-1]`?

A) ming

B) min

C) mmi

D) gram

Part 2: NumPy `nditer`

11. By default, what iteration order does `numpy.nditer` use when iterating over an array?

A) Always C-order (row-major).

B) Always Fortran-order (column-major).

C) It matches the memory layout of the array to maximize efficiency.

D) It iterates in alphabetical order.

12. If you want to modify the elements of an array while iterating over it with `numpy.nditer`, which parameter must you pass to the iterator?

A) `flags=['readwrite']`

B) `op_flags=['readwrite']`

C) `modify=True`

D) `mode='w'`

13. Given `arr = np.array([[1, 2], [3, 4]])`, what sequence of numbers is printed if you iterate using `np.nditer(arr, order='F')`?

A) 1, 2, 3, 4

B) 1, 3, 2, 4

C) 4, 3, 2, 1

D) 3, 4, 1, 2

14. What does the `flags=['external_loop']` parameter do when passed to `np.nditer`?

A) It skips the innermost loop and only iterates over the outer arrays.

B) It yields 1D arrays of values rather than individual elements whenever possible.

C) It allows Python to execute the loop outside of the Global Interpreter Lock (GIL).

D) It loops over the array infinitely until a break condition is met.

15. Can `numpy.nditer` iterate over two arrays simultaneously?

A) Yes, but only if they have the exact same shape and memory layout.

B) Yes, it can iterate over multiple arrays simultaneously as long as they are broadcastable to the same shape.

C) No, `nditer` is strictly designed for single-array iteration.

D) Yes, but the second array will automatically be flattened regardless of its shape.

16. If `numpy.nditer` needs to change the data type of the elements during iteration (data type casting), which flag must be provided?

A) `flags=['cast']`

B) `flags=['buffered']`

C) `op_flags=['cast_all']`

D) `dtype_change=True`

17. What is the fundamental difference between the assignment operator (`=`) and a shallow copy (`copy.copy()`) when applied to a list?

A) The assignment operator creates a new list in memory, while shallow copy creates a reference to the existing list.

B) The assignment operator creates a reference to the existing list, while a shallow copy creates a new top-level list object.

C) There is no difference; they behave exactly the same way.

D) Shallow copy duplicates all nested elements, while assignment only duplicates the top level.

18. Given the code:

```
None
import copy
a = [[1, 2], 3, 4]
b = copy.copy(a)
a[0][0] = 99
```

What is the value of `b[0][0]`?

A) 1

B) 99

C) raises an `AttributeError`

D) `[1, 2]`

19. Given the code:

```
None
import copy
x = [10, 20, 30]
y = copy.deepcopy(x)
x[0] = 100
```

What is the value of `y[0]`?

A) 100

B) 10

C) `None`

D) 0

20. Which standard library module must be imported to use the `deepcopy()` function?

A) `sys`

B) `deepcopy`

C) `copy`

D) `clone`

21. Given the dictionary `d1 = {'keys': [1, 2, 3]}`, what happens if you execute `d2 = d1.copy()` and then `d2['keys'].append(4)`?

A) `d1['keys']` remains `[1, 2, 3]` and `d2['keys']` becomes `[1, 2, 3, 4]`.

B) Both `d1['keys']` and `d2['keys']` become `[1, 2, 3, 4]`.

C) A `TypeError` is raised because dictionaries cannot be copied.

D) `d1['keys']` becomes `[1, 2, 3, 4]` and `d2['keys']` becomes `[1, 2, 3]`.

22. If you are copying a completely "flat" list containing only immutable elements (e.g., integers like `[1, 2, 3]`), how does `copy.deepcopy()` behave compared to `copy.copy()`?

A) `deepcopy()` will fail and raise a `TypeError` on flat lists.

B) Functionally, there is no difference in the resulting object's behavior, though `deepcopy()` takes slightly more overhead.

C) `deepcopy()` will turn the integers into float values.

D) `copy.copy()` creates references to the integers, but `deepcopy()` creates entirely new integer objects in memory.

23. How does `copy.deepcopy()` handle objects that contain circular/cyclic references (e.g., a list that appends itself)?

A) It enters an infinite loop and crashes with a `RecursionError`.

B) It ignores the cyclic reference and leaves a `None` in its place.

C) It keeps a "memo" dictionary of already copied objects to recreate the cycle without infinite recursion.

D) It raises a `ValueError: cyclic reference detected`.

24. Given `original = [(1, 2), [3, 4]]` and `shallow = copy.copy(original)`, what happens if you try to mutate the tuple inside `original`?

A) The tuple in `shallow` updates automatically.

B) Tuples are immutable, so you cannot modify `(1, 2)` in place; you would have to replace the whole tuple.

C) The tuple in `shallow` is destroyed.

D) A `DeepCopyWarning` is raised.

25. Which magic (dunder) method can you implement in a custom Python class to explicitly define how `copy.deepcopy()` should behave when copying instances of that class?

A) `__copy__`

B) `__clone__`

C) `__deepcopy__`

D) `__init_copy__`

26. In Python, using the list constructor (e.g., `new_list = list(old_list)`) produces which type of copy?

A) Deep copy

B) Shallow copy

C) Direct reference (alias)

D) Read-only copy

27. Given `a = [1, 2, 3]` and `b = a`, what is the result of the expression `a is b`?

A) `False`

B) `True`

C) `None`

D) Raises a `SyntaxError`

28. Given `a = [[1, 2], [3, 4]]` and `b = copy.deepcopy(a)`, what is the result of the expression `a[0] is b[0]`?

A) `True`

B) `False`

C) `None`

D) `1`

29. Which of the following operations does NOT create a shallow copy of a list `lst`?

A) `lst[:]`

B) `list(lst)`

C) `lst.copy()`

D) `lst2 = lst`

30. Why is deep copying generally slower than shallow copying?

A) Because deep copying has to serialize the object to JSON and back.

B) Because deep copying requires the OS to allocate protected memory blocks.

C) Because deep copying must recursively traverse all nested objects and create independent copies of mutable elements.

D) Because shallow copy is written in Python, while deep copy uses an external API.